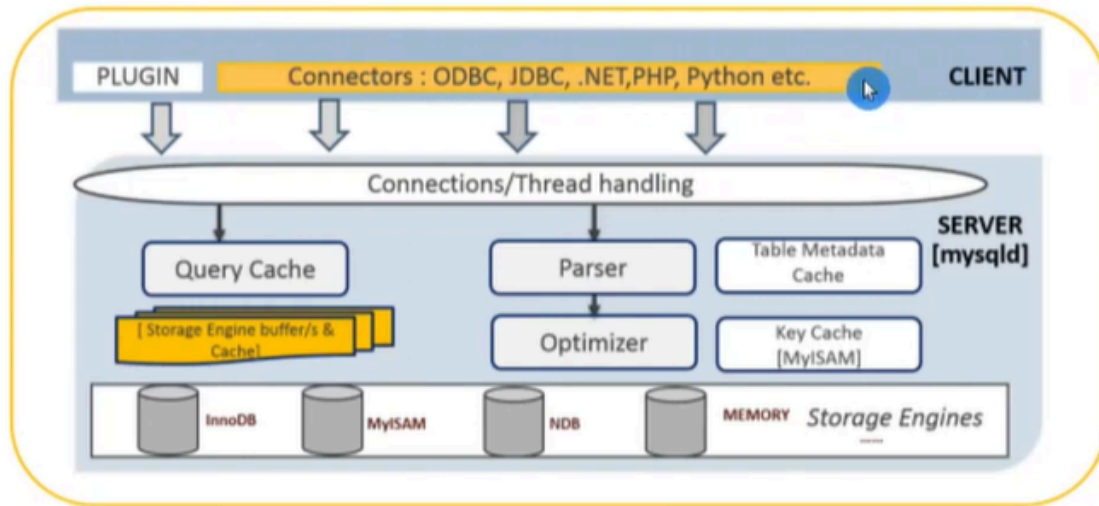


MySQL's Logical Architecture



Because MySQL
8 is Good 😊

- **SQL DML**
 - Window Functions, Recursive/Non-Recursive CTEs, Derived Tables,...
- **SQL DDL**
 - Instant ADD/RENAME Column,...
- **INDEXES**
 - Invisible/Descending/Functional Indexes,...
- **FUNCTIONS**
 - Regexp_%, UUID,...
- **JSON**
 - JSON_%, Sorting, In-place

Because MySQL
8 is Better 😊

- **GIS**
 - Spatial Reference Systems, ST functions, Geography Support
- **Character Sets and Collations**
 - UTF-8, Unicode 9.0, ...
- **Information Schema**
 - Views over Data Dictionary, New Views, ...
- **Performance Schema**
 - Indexes, Error handling, Data Locks, Latency histograms
- **Show**
 - Hidden columns, Index information



Because MySQL
8 is Great 😊

Optimizer

- Histograms, Hints,

InnoDB

- Redo Log, Encryption, ...

Data Dictionary and Upgrade

- Transactional DD, Meta Data in InnoDB, Auto Upgrade

Replication / Group Replication

- Binlog Encryption, Atomic DDL Recovery / Savepoints, Switchovers, Auto-rejoin

Security, Shell, Router and many more [features](#).

Pros of Upgrading to New Version

- Rich features (Business specific needs)
- Technical advantages (QPS, Throughput, Administration)
- Avoid multiple version upgrades
- Bug fixes
- Security and Compliance
- Community and Vendor support

Cons of Upgrading to New Version



Risks of hitting
unknown
issues



Different
behavior and
results



Slower
throughput
due to

sql_mode,
Optimizer, Charset,
etc



Amount of
time and
material
needed for
upgrade
project



Downtime that
is scheduled to
happen



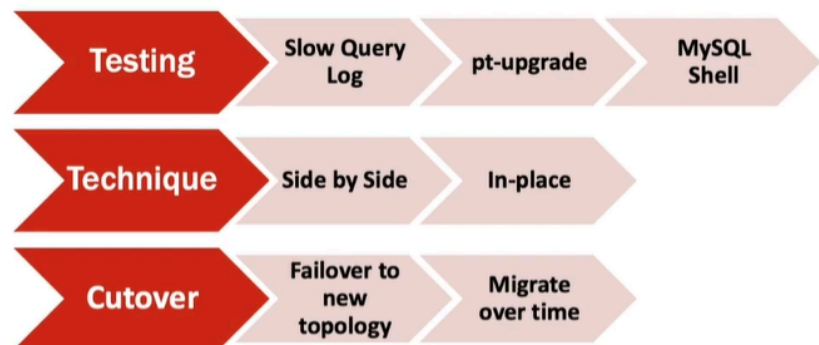
Downgrade
from MySQL
8.0 to MySQL
5.7 is not
supported.

Including [minor
version
downgrades](#) within
MySQL 8.0

Traditionally pre-V8

- **Major versions only (Ascending Order)**
 - 5.1 → 5.5
 - 5.5 → 5.6
 - 5.6 → 5.7
 - Set innodb_fast_shutdown=0
 - Stop existing MySQL server
 - Change binaries to new version
 - Adjust config (my.cnf) to new version
 - Start MySQL server
 - Run mysql_upgrade(Time Consuming)
 - Restart MySQL Server (Additional Downtime)

MySQL 8.0 Upgrade Process



Before You Even Start Testing

- Review release notes
- Changes in defaults
- Features of new version
- Removed syntax
- Deprecated functions (GIS)
- Data restrictions
- New keywords
- Changed behavior

Testing Phase

Setup a new testing environment

- Setup slaves as replication
 - Keep it for a month on no-access server
 - Try read and batch testing
 - Verify schema / connectivity compatibility
- Use ProxySQL (mirroring, traffic routing)
- Inspect slow query log

Build independent cluster

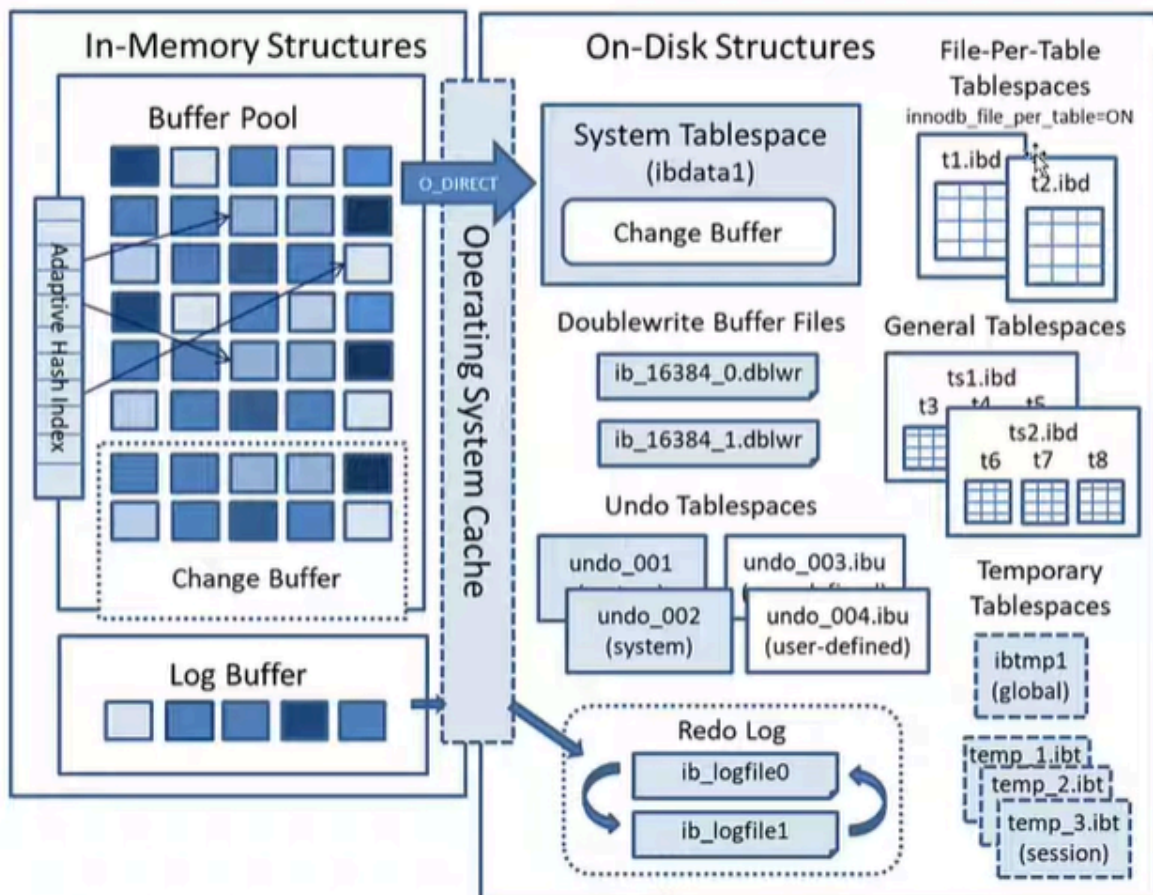
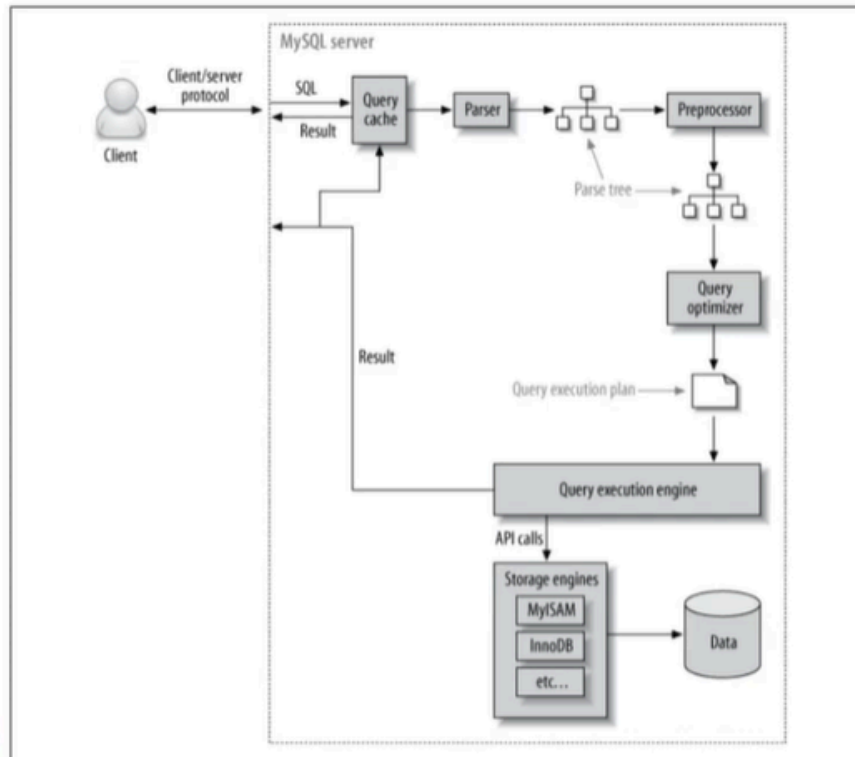
- Query testing
 - Capture slow queries and digest (pt-query-digest)
 - pt-upgrade
- Application testing
 - Verify results

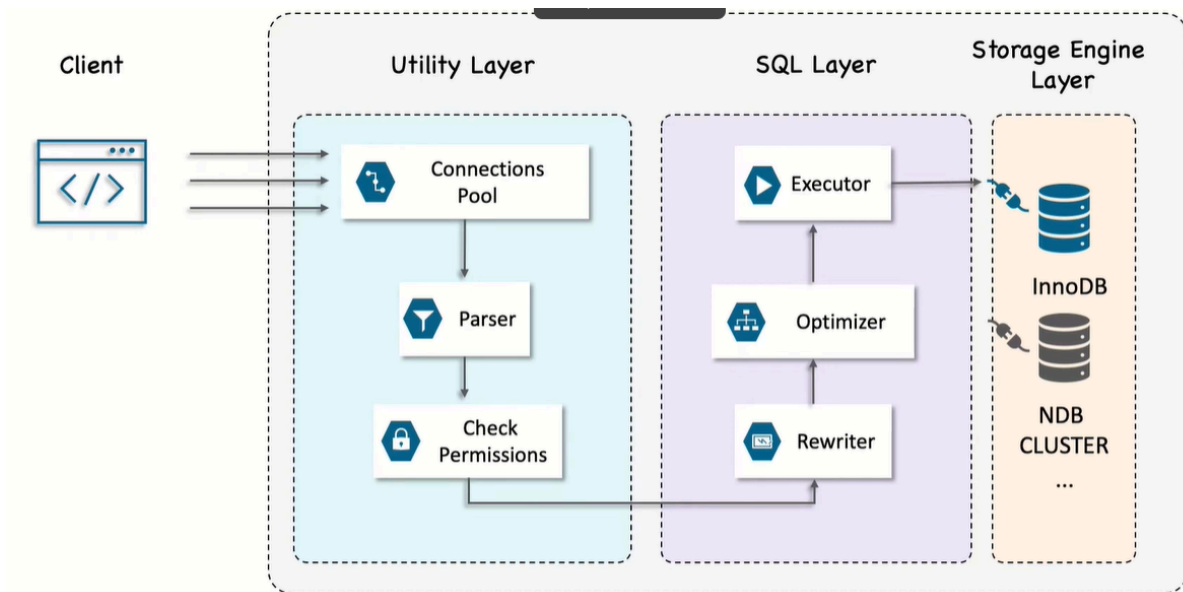
MySQL Upgrade 5.7 to 8.0 (In-place)

- **5.7 to 8.0**
 - Stop existing MySQL server
 - Change binaries to new version
 - Adjust config (my.cnf) to new version
 - Start MySQL server
 - Automatic upgrade
 - New mysqld options
 - --upgrade=AUTO (default)
 - --upgrade=NONE (errors if anything needs upgrade)
 - --upgrade=MINIMAL (to avoid DDL on some system tables)
 - --upgrade=FORCE
 - Eliminates issues with metadata using transactional data dictionary

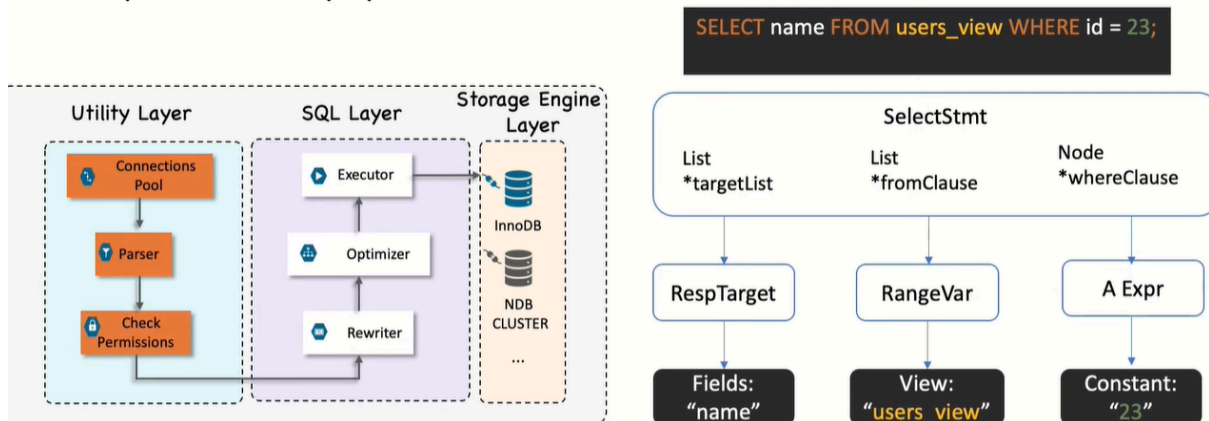
MySQL Upgrade 5.7 to 8.0 (Replication & Rolling Upgrade)

- **5.7 to 8.0**
 - Stop existing MySQL Slave server
 - Change binaries to new version
 - Adjust config (my.cnf) to new version
 - Start MySQL server
 - Automatic upgrade
 - New mysqld options
 - --upgrade=AUTO (default)
 - --upgrade=NONE (errors if anything needs upgrade)
 - --upgrade=MINIMAL (to avoid DDL on some system tables)
 - --upgrade=FORCE
 - Eliminates issues with metadata using transactional data dictionary
- Repeat above for minimum of 2 slaves and promote one as master





The parser and the preprocessor



The Query Optimizer

